

Document e-Soleau – Langage Interne de Raisonnement pour Modèles de Langage

1. Objet de l'invention et domaine technique

La présente description concerne le domaine des **modèles de langage de grande taille** (LLM, *Large Language Models*) basés sur des réseaux de neurones profonds, utilisés pour la génération de texte, le raisonnement et l'assistance interactive.

Plus précisément, elle décrit :

- des **procédés d'apprentissage** et d'utilisation d'un **langage interne de raisonnement discret** (*IR*, pour *Internal Reasoning*), distinct du langage naturel ;
- des **architectures de modèles** mettant en œuvre un **goulot d'étranglement discret** (dit *air-gap*) entre l'encodage de l'entrée et la génération de la réponse ;
- des **architectures à deux modèles** reposant sur un **langage abstrait intermédiaire** pour la traduction bidirectionnelle entre langage naturel et langage interne.

L'objectif global est de **dépasser les limites des approches de Chain-of-Thought (CoT)** en langage naturel, et plus généralement des LLM qui raisonnent uniquement dans l'espace des phrases humaines.

2. Problématiques et limites de l'état de la technique

2.1 Limitations des LLM sans Chain-of-Thought

Les LLM actuels, sans CoT explicite, présentent les limites suivantes :

1. **Opacité des raisonnements** Les raisonnements sont réalisés dans des états latents continus de haute dimension, non interprétables. Il est difficile voire impossible d'extraire une trace causale du cheminement interne.
2. **Manque de contrôlabilité** L'absence de trace de raisonnement explicite rend la vérification, la correction ou la contrainte des étapes intermédiaires très difficile (hallucinations, erreurs silencieuses).
3. **Conflation entre “comprendre” et “mimer”** Le modèle peut produire une réponse correcte par simple mappage entrée → sortie, sans structure interne de raisonnement réutilisable.

2.2 Limitations des LLM avec Chain-of-Thought en langage naturel

Les approches dites **CoT (Chain of Thought)** consistent à faire produire au modèle une suite d'explications en langue naturelle avant la réponse finale (par exemple “Réfléchis étape par étape”).

Ces approches améliorent souvent les performances, mais présentent plusieurs limites :

1. Langage humain non optimal pour les machines

- Le langage naturel est **verbeux, ambigu et sur-redondant**.
- Il est conçu pour que des humains se comprennent, pas pour l'optimisation computationnelle.

- Le raisonnement en CoT est donc contraint par une grammaire et un vocabulaire non adaptés à une représentation dense de l'information.

2. Coût computationnel élevé

- Produire de longues chaînes CoT augmente le nombre de tokens générés et donc le coût.
- Beaucoup de tokens CoT ne contribuent pas directement à la décision finale.

3. CoT non nécessairement causal

- Le modèle peut apprendre à produire des explications post-hoc qui “semblent plausibles” mais ne reflètent pas le chemin réel de calcul interne.
- La chaîne CoT n'est pas forcément **causalement nécessaire** : perturber le texte explicatif ne suffit pas à dégrader la qualité de la réponse.

4. Difficulté d'optimisation directe

- Les CoT sont générés dans le même espace que les réponses, avec les mêmes objectifs de probabilité de tokens.
- Il est difficile d'imposer que les CoT maximisent réellement la capacité de raisonnement, plutôt que simplement la vraisemblance textuelle.

2.3 Problème du contexte et de la gestion d'information

Les LLM sont également limités par :

- La **fenêtre de contexte** (nombre maximal de tokens pris en compte).
- L'incapacité à **compresser sémantiquement** de grands volumes d'informations dans une représentation interne structurée, réutilisable sur plusieurs tâches.

Sans mécanisme explicite de **langage interne compact**, les modèles doivent soit :

- relire en permanence de longues séquences de texte,
- soit compter sur des états latents continus non contrôlables.

2.4 Lacunes de l'art antérieur

À ce jour, et à notre connaissance, l'état de la technique ne décrit pas la combinaison suivante :

- de mécanisme imposant un **langage interne discret**, distinct du langage naturel,
- **causalement nécessaire** (goulot d'étranglement sans contournement)
- et **entraîné explicitement** pour :
 - compresser sémantiquement des entrées en langage naturel,
 - servir de support de raisonnement,
 - puis être re-traduit en langage naturel.

Il existe des travaux sur :

- la quantification vectorielle pour la compression,
- les autoencodeurs de séquences,
- les CoT textuels,
- et la distillation de raisonnements,

mais ceux-ci ne décrivent pas une **architecture systématique combinant** :

1. un **air-gap discret** obligatoire,
 2. un **langage interne de raisonnement** émergent,
 3. et des **procédés d'apprentissage en plusieurs phases** alignés sur le raisonnement.
-

3. Définitions des concepts techniques introduits

Pour la suite, on introduit les définitions suivantes :

1. **Langage Interne de Raisonnement (LIR ou IR)** Ensemble fini de symboles discrets (codes, jetons) utilisés exclusivement comme **support de raisonnement interne** d'un modèle.
 - Ces symboles ne sont pas visibles à l'utilisateur final.
 - Ils ne sont pas directement des mots d'une langue naturelle.
 - Ils sont appris automatiquement pendant l'entraînement.
 2. **Jetons de Raisonnement Interne (JRI)** Séquence ordonnée de symboles du LIR produite à partir d'une entrée et utilisée comme unique vecteur de communication entre un module encodeur et un module décodeur.
 3. **Architecture à Gap d'Air (air-gap)** Architecture de réseau où il **n'existe aucun chemin d'information direct** entre l'encodage de l'entrée et la génération de la sortie, en dehors d'un **goulot d'étranglement discret** (les JRI).
 - Le modèle est contraint d'encoder toute l'information pertinente dans les JRI.
 4. **Langage Abstrait (LA)** Variante de LIR utilisée dans une architecture à deux modèles :
 - un modèle de traduction langue naturelle → LA,
 - un modèle de traduction LA → langue naturelle. Le LA peut être partagé entre plusieurs tâches et domaines.
 5. **Raisonnement IR-CoT** Procédé dans lequel un modèle :
 - reçoit une entrée en langage naturel,
 - la convertit en JRI (langage interne),
 - réalise des opérations de raisonnement dans cet espace interne,
 - puis génère soit :
 - une chaîne de type CoT en langage naturel,
 - soit une réponse finale, soit les deux.
-

4. Solutions proposées et procédés associés

Les solutions décrites ci-après sont **indépendantes** mais **combinables**. Chaque solution est formulée comme un **procédé technique** pouvant, en principe, servir de base à une protection par brevet.

4.1 Procédé 1 : Architecture Air-Gap avec Langage Interne de Raisonnement (IR-CoT)

4.1.1 Principe général

Le premier procédé consiste à :

- insérer un **goulot d'étranglement discret** (séquence de JRI) au cœur d'un modèle de langage de type transformeur ;
- le configurer de sorte que ce goulot soit le **seul chemin** entre l'entrée et la sortie ;
- entraîner ce goulot pour en faire un **langage interne de raisonnement** émergent, utilisé ensuite pour des tâches de raisonnement de haut niveau.

4.1.2 Architecture de base

Le procédé met en œuvre :

1. Un **module encodeur** (par exemple un empilement de couches de transformeur) recevant :
 - une séquence d'entrée en langage naturel (texte),
 - éventuellement d'autres modalités (structure, code, etc.).
2. Un **module de quantification/discretisation** :
 - prenant comme entrée les représentations continues produites par l'encodeur,
 - projetant ces représentations sur un **dictionnaire de codes** (codebook),
 - produisant une séquence de **JRI** (indices de code) de longueur fixe ou bornée.
3. Un **module décodeur** prenant exclusivement comme entrée :
 - la séquence de JRI,
 - et générant une séquence en langage naturel (réponse, explication, CoT, etc.).
4. Une **contrainte structurelle** :
 - aucune connexion directe (résiduelle, skip, concaténation) ne permet aux représentations continues de l'encodeur d'atteindre le décodeur ;
 - les seules informations disponibles pour le décodeur sont les JRI.
 - les flux attentionnels croisés sont autorisés uniquement **via les JRI**, ce qui élimine tout contournement implicite de l'air gap.

4.1.3 Tests de causalité de l'air gap

- **Scrambling / permutation** des JRI doit dégrader significativement les performances.
- **Masquage / drop** des JRI doit empêcher le décodeur de répondre correctement.
- Ces tests démontrent que le canal JRI est **causalement nécessaire** et non décoratif.

4.1.4 Procédé d'apprentissage (en plusieurs phases)

Le procédé d'apprentissage typique comprend au moins deux phases distinctes :

Phase 1 – Apprentissage sémantique du LIR

1. Fournir au modèle des paires de textes en langage naturel :
 - soit (segment A, segment B) d'une même histoire ou dialogue,
 - soit (texte, reformulation, continuation).

2. Pour chaque paire :

- encoder A,
- produire des JRI via le module de quantification,
- entraîner le décodeur à **prédire B** à partir des JRI.

3. La fonction de coût favorise :

- la prédiction correcte de B,
- et donc l'émergence de JRI capturant l'**état sémantique** (contexte, entités, intentions).

4. Optionnellement, des **pertes auxiliaires** peuvent être ajoutées :

- pertes de type débruitage (reconstruction de parties masquées de A),
- pertes contrastives (JRI similaires pour textes sémantiquement proches).

Phase 2 – Spécialisation au raisonnement

5. Fournir au modèle des exemples de tâches de raisonnement :

- problèmes logiques, mathématiques, questions/réponses, etc.

6. Pour chaque exemple :

- encoder l'énoncé en texte,
- produire des JRI,
- entraîner le décodeur à **prédire la réponse** (et éventuellement une chaîne CoT textuelle) à partir des JRI.

7. Les paramètres du module de quantification et du codebook peuvent être :

- soit figés (langage interne stabilisé),
- soit affinés avec un taux d'apprentissage réduit.

4.1.5 Variantes du procédé 1

- **Variante IR + CoT textuel** : le décodeur génère d'abord une séquence CoT en langage naturel, puis la réponse finale.
- **Variante sans CoT textuel** : le décodeur génère directement la réponse ; le LIR constitue la seule trace de raisonnement.
- **Variante multi-domaine** : même LIR partagé entre texte, code et autres modalités.

4.2 Procédé 2 : Architecture à deux modèles via Langage Abstrait (LA)

4.2.1 Principe général

Le deuxième procédé découple :

- la **traduction** entre langage naturel et langage interne,
- du **raisonnement** dans ce langage interne.

On introduit un **Langage Abstrait (LA)** comme pivot entre plusieurs systèmes.

4.2.2 Architecture

Le procédé met en œuvre :

1. Un **premier modèle** (Encodeur LA) qui :
 - reçoit une entrée en langage naturel,
 - encode cette entrée en représentations continues,
 - les quantifie dans un **Langage Abstrait** (séquence de codes ou JRI).
2. Un **second modèle** (Décodeur LA) qui :
 - reçoit la séquence LA,
 - produit soit :
 - une réponse en langage naturel,
 - une explication en langage naturel,
 - ou une structure de sortie (par ex. code, JSON, etc.).
3. Optionnellement, un **troisième module** interne ou externe, spécialisé dans le **raisonnement sur le LA** :
 - capable de transformer une séquence LA en une autre séquence LA avant la décodification,
 - agissant comme “moteur de raisonnement” au dessus du Langage Abstrait.

4.2.3 Procédé d'apprentissage

1. **Apprentissage traductionnel** :
 - Entraîner le premier modèle à produire un LA qui permet au second modèle de reconstruire :
 - la suite du texte,
 - une paraphrase,
 - ou une réponse correcte.
2. **Apprentissage du raisonnement en LA** :
 - Entraîner le module de raisonnement (interne ou externe) sur des paires (LA_entrée, LA_sortie) correspondant à des transformations logiques ou mathématiques.
3. **Compatibilité multi-modèle** :
 - Le LA est défini de manière à pouvoir être réutilisé par plusieurs encodeurs ou décodeurs (multi-tâches, multi-domaines).
 - Les signaux d'apprentissage peuvent combiner reconstruction, prédiction de continuation et feedback humain/RL, à condition de conserver le LA comme unique vecteur de communication (pas de chemins directs ou de biais de contournement).

4.2.4 Intérêt du procédé 2

- Permet de **séparer les responsabilités** :
 - comprendre le langage humain,
 - manipuler des structures abstraites,
 - générer une réponse lisible.

- Offre une base pour créer des **pipelines modulaires** où le même LA sert :
 - à l'interfaçage avec différentes langues,
 - ou avec des systèmes symboliques.
-

4.3 Procédé 3 : Raisonnement parallèle IR + CoT en langage naturel

4.3.1 Principe général

Le troisième procédé combine :

- un **flux de raisonnement interne** en LIR (IR),
- un **flux de raisonnement externe** en CoT textuel.

Idée : laisser le modèle apprendre **où et quand** utiliser :

- le langage naturel (externe, explicatif),
- le langage interne discret (interne, dense).

4.3.2 Architecture

1. À partir d'une entrée en langage naturel, le modèle produit :

- des activations de transformeur classiques,
- **et** une séquence de JRI (LIR) via un module de quantification interne.

2. Le modèle dispose de deux “pistes” de raisonnement :

- une piste **CoT textuelle**, où certaines couches génèrent ou conditionnent des tokens de CoT,
- une piste **IR**, où d'autres couches lisent et écrivent des JRI.

3. Les deux pistes peuvent :

- s'informer mutuellement (via attention croisée),
- ou être contraintes (par exemples pertes auxiliaires) à rester cohérentes.
- La cohérence est contrôlée par des pertes de consistance (IR ↔ CoT) et des tests de dégradation : si l'on masque ou mélange l'IR, la chaîne CoT doit se détériorer, ce qui prouve que la piste IR n'est pas décorative.

4.3.3 Procédé d'utilisation

- En mode **explicable** :
 - le modèle produit simultanément :
 - une chaîne CoT en langage naturel,
 - et une trace IR.
- En mode **optimisé** :
 - le modèle peut utiliser majoritairement l'IR,
 - et limiter le CoT textuel à des résumés, ce qui réduit le coût en tokens.

4.3.4 Intérêt du procédé 3

- Faire coexister :
 - **raisonnement humainement lisible** (CoT),
 - **raisonnement machine-optimisé** (IR).
 - Étudier empiriquement quelles parties du raisonnement le modèle préfère confier à l'IR versus au langage naturel.
-

5. Avantages techniques des procédés proposés

Les procédés décrits présentent plusieurs avantages par rapport à l'état de la technique :

1. Séparation nette entre langage externe et langage interne

- Permet d'optimiser le langage interne pour la densité d'information et la réutilisabilité, indépendamment des contraintes du langage humain.

2. Causalité et inspectabilité du raisonnement

- Dans l'architecture air-gap, les JRI sont causalement nécessaires : les perturber dégrade le résultat.
- Cette propriété rend le raisonnement **auditable** et **analysable**.

3. Potentiel de compression sémantique

- Les JRI peuvent encoder l'"état" d'un problème sous forme courte, ce qui facilite :
 - la gestion de grands contextes,
 - le transfert de connaissances entre tâches.

4. Modularité

- Le Langage Abstrait (LA) permet de coupler différents modèles et systèmes autour d'un langage pivot.
- Des moteurs de raisonnement spécialisés peuvent être insérés au centre (LA → LA).

5. Compatibilité avec les approches CoT existantes

- Le procédé 3 permet de combiner IR et CoT textuel, au lieu d'opposer les deux.
 - Les modèles existants peuvent être étendus avec une piste IR sans perdre leurs capacités CoT.
-

6. Conclusion et précisions

Le présent document décrit :

- des **concepts nouveaux** :
 - Langage Interne de Raisonnement (LIR / IR),
 - Jetons de Raisonnement Interne (JRI),
 - Architecture à Gap d'Air,
 - Langage Abstrait (LA),

- Raisonnement IR-CoT ;

- et plusieurs **procédés techniques** :

1. un **procédé d'apprentissage et d'utilisation d'une architecture air-gap** où un LIR discret est la seule médiation entre entrée et sortie ;
2. un **procédé d'architecture à deux modèles** utilisant un LA pivot pour traduire et raisonner ;
3. un **procédé de raisonnement parallèle IR + CoT**, combinant langue naturelle et langage interne.

Ces procédés peuvent être :

- appliqués seuls,
- combinés dans un même système,
- ou utilisés comme briques de base pour des systèmes plus complexes (agents, systèmes multi-modèles, assistants spécialisés, etc.).

Les détails d'implémentation (taille des modèles, nature exacte des couches, jeux de données, résultats quantitatifs) pourront être précisés dans des annexes ou des versions ultérieures. Ils ne changent pas la nature des procédés revendiqués ici, qui portent sur :

- la **structure générale de l'architecture**,
- le **rôle causal du langage interne discret**,
- et les **schémas d'apprentissage multi-phase** visant à faire émerger et exploiter ce langage interne pour le raisonnement.

7. Usages industriels envisagés

Les procédés s'appliquent à des modèles de langage (texte ou multimodaux) de toute taille et architecture (dense, Mixture-of-Experts, avec ou sans CoT explicite), en déploiement cloud ou embarqué (téléphone, systèmes contraints).

- **Efficacité et embarqué** : la compression sémantique via IR peut réduire le coût en tokens et la latence, utile pour l'exécution locale (privacy/offline) et pour des contraintes énergétiques.
- **Sûreté et traçabilité** : l'air gap et les tests de causalité (scramble/drop IR) apportent un signal de dépendance fonctionnelle au canal IR. Cela ne constitue pas une explicabilité textuelle ; l'IR est émergent et nécessite des sondes dédiées pour être interprété, mais le caractère causal et testable du canal facilite l'audit de l'usage effectif de l'IR.
- **Modularité** : un langage pivot LA/IR permet de chaîner plusieurs modèles (cloud + edge) ou moteurs spécialisés tout en maintenant un canal discret central.

Ces usages restent non limitatifs et n'altèrent pas la portée des procédés décrits.

ANNEXE – RÉSUMÉ DES EXPÉRIMENTATIONS ET LIEN AVEC LES PROCÉDÉS DÉCRITS

1. Objet de l'annexe et lien avec le texte principal

La présente annexe a pour objet de **documenter l'état actuel des expérimentations** menées autour :

- de l'architecture à **air gap** avec langage interne discret (**IR**, pour *Internal Reasoning*), et

- de son utilisation pour implémenter un **raisonnement de type CoT interne** (*IR-CoT*), distinct du langage naturel.

Elle ne crée pas de nouveaux procédés par rapport au texte principal de l'e-Soleau, mais :

1. **illustre** la mise en œuvre concrète des procédés décrits (architectures, flux de données, contraintes d'entropie, etc.) ;
2. **corrobore la faisabilité** technique des idées (langage interne discret émergent, nécessité causale de l'IR, dépendance à la capacité du modèle) ;
3. **soutient la cohérence de l'invention** en montrant une progression logique de versions expérimentales qui ciblent précisément les problématiques identifiées (limitations des CoT classiques, gestion de l'entropie, rôle de l'IR, etc.).

Les expériences décrites ci-dessous doivent être considérées comme des **exemples de mise en œuvre non limitatifs** des procédés décrits dans le corps principal de l'e-Soleau.

2. Chronologie synthétique des versions expérimentales

2.1. Versions V10 à V12 – Preuve de concept de l'architecture à air gap

- **Objectif** : Vérifier que l'architecture à **air gap** avec un **canal discret IR** peut être utilisée comme médium de raisonnement *causalement nécessaire* sur des tâches simples (arithmétique, logique).
 - **Architecture** :
 - Entrée en langage naturel très simplifié ou en tokens symboliques.
 - Encodeur → **bouchon discret IR** (quantification vectorielle) → Décodeur.
 - Aucune connexion directe entrée → sortie contournant l'IR.
 - **Résultats principaux** :
 - Les modèles apprennent un **langage interne émergent** sous forme de codes IR.
 - **Scrambling** (permute) des séquences IR provoque une chute drastique des performances, ce qui démontre que les codes IR sont **causalement nécessaires** au raisonnement.
 - Sur des tâches mathématiques et logiques simples et à faible entropie, l'architecture fonctionne de façon robuste.
 - **Lien avec l'e-Soleau** : Ces versions démontrent la faisabilité des **procédés de raisonnement via un langage interne discret**, tel que décrit dans le texte principal (flux *Entrée* → *IR* → *Sortie*, contrainte d'air gap, émergence d'idéogrammes internes optimisés).
-

2.2. Version V16 – “Synthetic Trinity” (Math / Histoire synthétique / Chat synthétique)

- **Objectif** : Tester la **généralité de l'IR** sur plusieurs modalités textuelles tout en restant dans un cadre **syntétique à faible entropie**.
- **Données** :
 - Problèmes de mathématiques simples générés automatiquement.
 - Histoires courtes avec vocabulaire ~90 mots, structure grammaticale rigide.
 - Dialogues de type “chat” également générés de manière templatisée.

- **Architecture :**
 - Modèle ~22 M de paramètres.
 - IR de **32 tokens** (longueur fixe).
 - Entraînement en **deux phases** (pré-entraînement auto-encodeur, puis phase de raisonnement), conformément aux procédés décrits.
 - **Résultats :**
 - **100 % de précision** sur les trois tâches (Math / Story / Chat) avec un IR de 32 tokens.
 - Les codes IR sont stables, réutilisés et interprétables à un niveau symbolique simple.
 - **Lien avec l'e-Soleau :** V16 illustre que la combinaison :
 - architecture à air gap,
 - IR discret,
 - et entraînement en deux phases permet d'implémenter un **langage de pensée interne** efficace sur des domaines synthétiques. Cela renforce la validité des procédés revendiqués pour la partie "basse entropie".
-

2.3. Version V17 – Passage au langage réel (TinyStories + Math)

- **Objectif :** Vérifier si les procédés validés sur données synthétiques (V16) s'étendent à des textes en **langage naturel réel** (plus grande entropie).
- **Données :**
 - Corpus TinyStories (histoires en anglais réel, vocabulaire ~3000, grammaire libre).
 - Problèmes mathématiques synthétiques similaires à V16.
- **Architecture :**
 - Toujours un modèle ~22 M de paramètres.
 - IR de 32 tokens puis 64 tokens (bouchon "relâché").
 - Phase 1 : **auto-encodage exact** (reconstruction mot à mot).
 - Phase 2 : raisonnement (prédiction de la suite ou de réponses).
- **Résultats :**
 - Phase 1 : l'auto-encodeur parvient à reconstruire raisonnablement le texte d'entrée.
 - Phase 2 : **échec complet** – 0 % de précision sur les tâches de raisonnement en présence de TinyStories, même avec IR étendu à 64 tokens.
- **Analyse :**
 - L'auto-encodage force l'IR à représenter les **formes de surface** (syntaxe, mots fonctionnels) au détriment de l'**état sémantique pertinent**.
 - La combinaison **faible capacité (22 M) + forte entropie du langage réel + objectif de reconstruction** conduit à une **saturation du canal IR**, qui ne parvient plus à porter l'information logique nécessaire.
- **Lien avec l'e-Soleau :** V17 met en évidence l'un des **problèmes centraux** que les procédés décrits dans le texte principal cherchent à résoudre :

- la difficulté des architectures standard (et des objectifs typiques) à gérer l'**entropie du langage naturel** dans un canal de pensée discret,
 - la nécessité de procéder à des **modifications de l'objectif d'entraînement** et à un **changement d'échelle** pour que le raisonnement par IR reste viable.
-

2.4. Version V17_ter – Objectif prédictif sur TinyStories + IR de 64 tokens

- **Objectif** : Tester si le problème est principalement dû à l'**objectif de reconstruction** ou à la **capacité du modèle**, en remplaçant la reconstruction par un **objectif prédictif sémantique**.
- **Données** :
 - TinyStories (langage réel, même distribution qu'en V17).
 - Problèmes de mathématiques synthétiques.
- **Architecture** :
 - Modèle toujours limité à ~22 M de paramètres.
 - IR de **64 tokens**.
 - Phase 1 : **Next Segment Prediction** (entrée segment 1 → IR → prédiction segment 2), au lieu de reconstruction exacte.
 - Phase 2 : fine-tuning de raisonnement (prédiction de réponses mathématiques et narratives à partir de l'IR).
- **Résultats** :
 - Phase 1 :
 - Baisse significative de la loss.
 - Perplexité ≈ 376 sur un vocabulaire d'environ 3000 mots après 15 époques → le modèle apprend effectivement à exploiter l'IR pour une tâche prédictive non triviale.
 - Phase 2 :
 - **Échec à 0 % de précision** sur les tâches de raisonnement (maths et story), malgré une Phase 1 réussie.
- **Analyse** :
 - Le passage à un objectif prédictif (plus aligné avec la capture de l'état sémantique) **ne suffit pas** à rendre le raisonnement par IR possible avec un modèle de 22 M de paramètres.
 - Cela fournit un **argument fort en faveur d'un goulot d'étranglement en capacité** :
 - le modèle parvient à apprendre une représentation prédictive du texte via l'IR,
 - mais ne dispose pas de suffisamment de paramètres pour :
 1. stabiliser un langage interne riche,
 2. apprendre une fonction de raisonnement non triviale au-dessus de ce langage.
- **Lien avec l'e-Soleau** : V17_ter soutient l'idée, au cœur des procédés décrits, qu'un **langage interne discret de type IR** est viable mais nécessite :
 - une **capacité minimale** pour gérer l'entropie du langage naturel,

- une **objectivation de l'objectif d'entraînement** (préférer des objectifs prédictifs/sémantiques à la reconstruction brute). Ces observations motivent directement les choix de V18 décrits dans le texte principal (changement d'échelle, élargissement du canal IR, affinage des objectifs).

3. Synthèse du lien entre les expérimentations et les procédés de l'e-Soleau

Les expériences V10 à V17_ter apportent un **soutien empirique** aux idées et procédés décrits dans le document principal de l'e-Soleau :

1. Validation du concept de langage interne discret (IR) et de l'air gap

- Les résultats initiaux (V10–V16) confirment qu'un modèle peut :
 - inventer un **langage interne émergent**,
 - l'utiliser de façon **causale** pour raisonner,
 - tout en restant distinct du langage naturel en entrée/sortie.
- Cela illustre les procédés revendiqués d'**encodage** → **IR** → **décodage** avec coupure stricte des chemins directs.

2. Mise en évidence des limites liées à l'entropie du langage et à la capacité

- V17 et V17_ter montrent que, sous capacité limitée (22 M) :
 - l'augmentation de l'entropie (langage réel) et
 - la nature de l'objectif d'entraînement (reconstruction vs prédiction) affectent directement la capacité de l'IR à porter un raisonnement fiable.
- Ceci justifie les éléments de procédé consistant à :
 - adapter l'**objectif d'entraînement** (préférer des tâches prédictives / “stateful”),
 - calibrer la **capacité du modèle** et la **taille du canal IR** en fonction de l'entropie des données.

3. Justification du passage à l'échelle (V18 et suivants)

- Le constat “V17_ter échoue malgré un objectif prédictif” conduit logiquement aux choix de V18 décrits dans le texte principal :
 - augmentation de la taille du modèle (~124 M),
 - augmentation de la taille du codebook et de la longueur IR,
 - maintien d'objectifs prédictifs plutôt que purement reconstructifs.
- Ces décisions sont directement motivées par les enseignements des versions précédentes et constituent une **mise en œuvre concrète** des procédés généraux décrits dans l'e-Soleau.
- V18 vise ainsi à tester la robustesse du canal IR sur langage réel, à vérifier la causalité du canal après montée en capacité, et à mesurer l'effet de l'élargissement IR sans figer une configuration unique.

4. Caractère non limitatif de l'annexe

Les versions V10 à V17_ter décrites ici :

- ne limitent pas la portée des procédés de l'invention,
- ne couvrent qu'un **sous-ensemble** des architectures, hyperparamètres, objectifs et jeux de données envisageables,
- servent **d'illustration** des principes généraux :
 - langage interne discret émergent (IR),
 - air gap et causalité du canal IR,
 - ajustement de la capacité et de l'objectif pour gérer l'entropie du langage naturel,
 - utilisation de l'IR comme support d'un CoT interne non humain (IR-CoT).

D'autres variantes (par ex. modèles plus grands, autres corpus, architectures à deux modèles, raisonnement IR parallèle au CoT classique, etc.) entrent dans le cadre des procédés décrits dans le texte principal et pourront être explorées ultérieurement sans sortir de l'invention telle que définie.